# SMART STORE | MICROSERVICES PROJECT BRIEF

Goal: Build a compact, API-only .NET microservices demo inspired by the AirlineBookingSystem layout.

## 1. Service responsibilities

| | | |
|---|---|---|
| **Catalog API** | **MongoDB** | CRUD products; retrieve product details for AI assistant. |
| **Orders API** | **Redis** | Create/get orders; Redis as primary demo store with persistence enabled. |
| **Payments API** | **PostgreSQL** | Simulate payments; store payment result and publish PaymentProcessed. |
| **Notifications API** | **SQL Server** | Store notification history after payment; no real email/SMS required. |

## 2. End-to-end workflow

- POST /orders receives ProductId and Quantity; checks the Catalog API synchronously and saves a Pending order in Redis.
- Orders publishes OrderPlaced; Payments consumes it, simulates success, saves a PostgreSQL record, and emits PaymentProcessed.
- Orders consumes PaymentProcessed to mark an order Paid; Notifications consumes it to insert a SQL Server notification.
- AI assistant: POST /catalog/assistant asks a configurable LLM to answer product questions using catalog data as context.

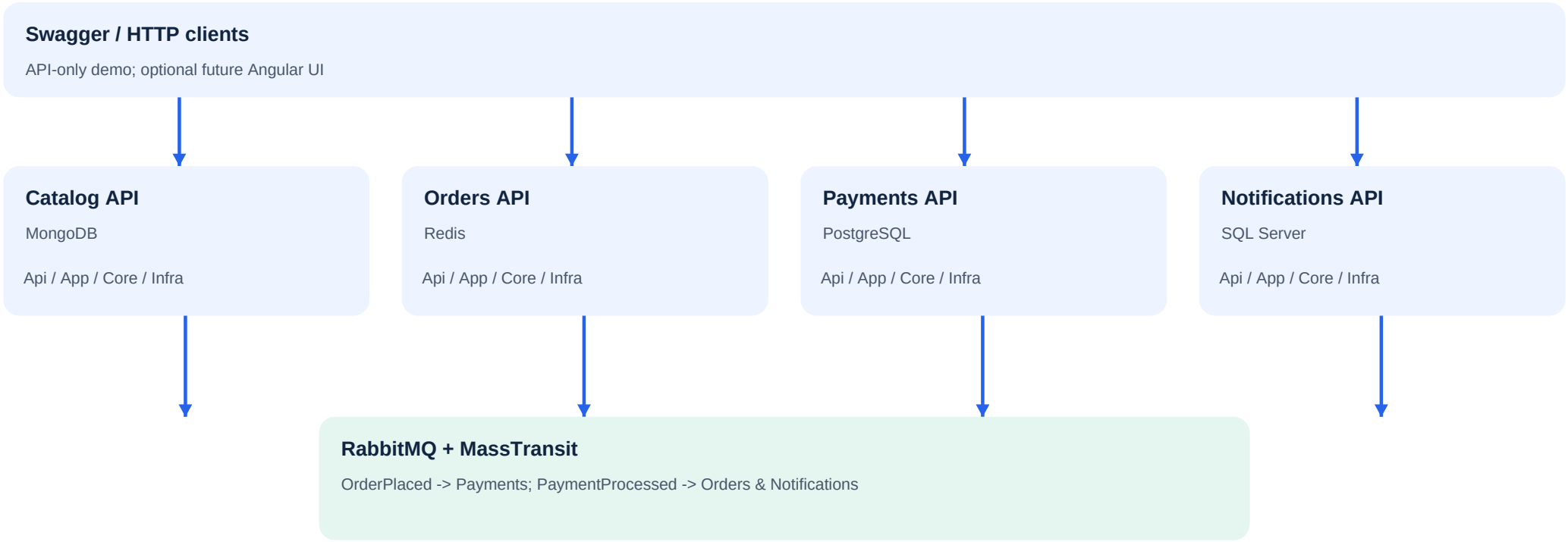
## 3. Required architecture and conventions

- .NET 8 APIs; each service contains Api / Application / Core / Infrastructure projects; shared BuildingBlocks for events.
- CQRS using MediatR; Core has entities and repository interfaces; Infrastructure contains storage integrations.
- MassTransit + RabbitMQ events; independent data ownership, Swagger/OpenAPI and environment-based configuration.

Reference: github.com/innovativeinstitution/AirlineBookingSystem | Scope: demo MVP, not production payments

# ARCHITECTURE DIAGRAM & DELIVERY CHECKLIST

Four independent API services, each with an isolated store and the same layered project structure.




**Event flow (illustrative):**

Orders publishes OrderPlaced > Payments consumes & publishes PaymentProcessed > Orders + Notifications consume

**AI product assistant (inside Catalog API)**

LLM via configurable provider + read-only catalog context

**Infrastructure (local)**

Docker Compose: APIs + 4 stores + RabbitMQ

## Deliverables / acceptance criteria

Solution + README + .env.example + Compose + seed data + curl/Postman examples + basic tests.

Demonstrate: create product -> place order -> payment event -> notification -> ask AI about products.